



Notch

A conservation layer for cross-chain cashflow-backed lending on Creditcoin. It decodes a real cashflow's financing capacity from a proven Ethereum transaction and enforces, on-chain, that no combination of lenders can finance more than that capacity.

BUIDL CTC 2026 Fall

Attestcoin Protocol

DeFi track

Testnet only

LIVE APP

notch-web-sooty.vercel.app

REPOSITORY

github.com/TheWeirdDee/notch

TESTS

87 / 87 passing

LEDGER

44 / 44 MATCH

One shared limit, enforced on-chain

Lenders on different chains can't see each other's commitments, so the same verified cashflow can look available to more than one of them at once and end up financed twice. Notch closes that gap: it decodes a cashflow's capacity from a proven, receipt-success Ethereum transaction — never a typed or cached number — and holds that capacity as one shared, depletable resource in a Creditcoin registry. Every lender draws from the same balance; a draw past what remains reverts on-chain, regardless of how many other lenders have already drawn.

THE PROBLEM

The same future cashflow can be financed twice by lenders who can't see each other's books — a documented, multi-billion-dollar failure mode in receivables finance. When the asset can also cross chains, an already-financed position can look untouched on the new chain: a lender on Creditcoin has no way to know what a lender on Ethereum has already committed against the same cashflow.

WHO THIS IS FOR

Lenders & underwriters

Assess a verifiable source and see exactly how much of it other lenders have already financed before committing funds.

Creditcoin's credit ecosystem

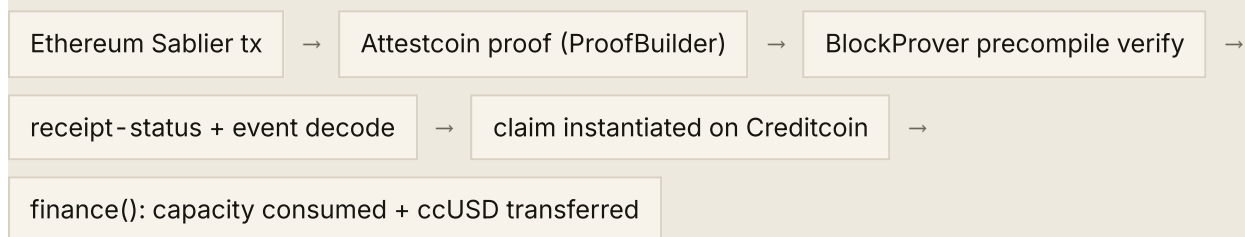
Build lending venues around one shared, enforceable capacity instead of isolated, unverifiable balance sheets.

WHAT SABLIER IS, AND WHY THE LOCK IS REAL

Sablier is a real, deployed streaming/lockup protocol on Ethereum — `SablierLockup v4.0`. Creating a stream locks actual ERC-20 tokens into that contract; nothing about the deposit is typed in or assumed. Attestcoin proves the stream's creation transaction happened and succeeded, and Notch decodes the locked `depositAmount` directly from that transaction's own `CreateLockupLinearStream` event — a real, third-party-held lock is what makes a claim's capacity real. Notch's contracts only ever *read* a Sablier lock this way; they never create, hold, or control Sablier funds.

Verify, finance, enforce

- 1 Verify.** Attestcoin proves a real Sablier lock's creation transaction is included and succeeded; the registry decodes the locked deposit straight from its `CreateLockupLinearStream` event, never a typed number.
- 2 Finance.** A lender calls `finance()`: ccUSD transfers to the borrower and capacity is consumed, atomically, in one Creditcoin transaction.
- 3 Enforce.** Every other lender sees the same remaining balance. A draw past what's left reverts on-chain, every time.



WHY ATTESTCOIN IS LOAD-BEARING, NOT DECORATIVE

`instantiateClaim` — the only function on `AttestedCashflowRegistry` that can ever create a claim — spends its first lines calling the native BlockProver precompile's `verifyAndEmit`, and reverts `VerificationFailed()` immediately if that call returns false, before touching anything else. Every check after it — receipt-success, event decode, source-contract allowlist, lock-shape rules — runs on data that only exists because that precompile call already succeeded.

There is no second path in: no owner-set override, no admin mint, no `onlyOwner` modifier anywhere in either contract — checked directly against the deployed source, not assumed. `venue` is `immutable`, and `setVenue` is a permanent, unconditional revert stub kept only so the call site still resolves.

Checkable in one read, not just asserted: `available()` is always `originalCapacity - financedCapacity`, and `originalCapacity` is only ever written inside `instantiateClaim`, after the verify step above passes.

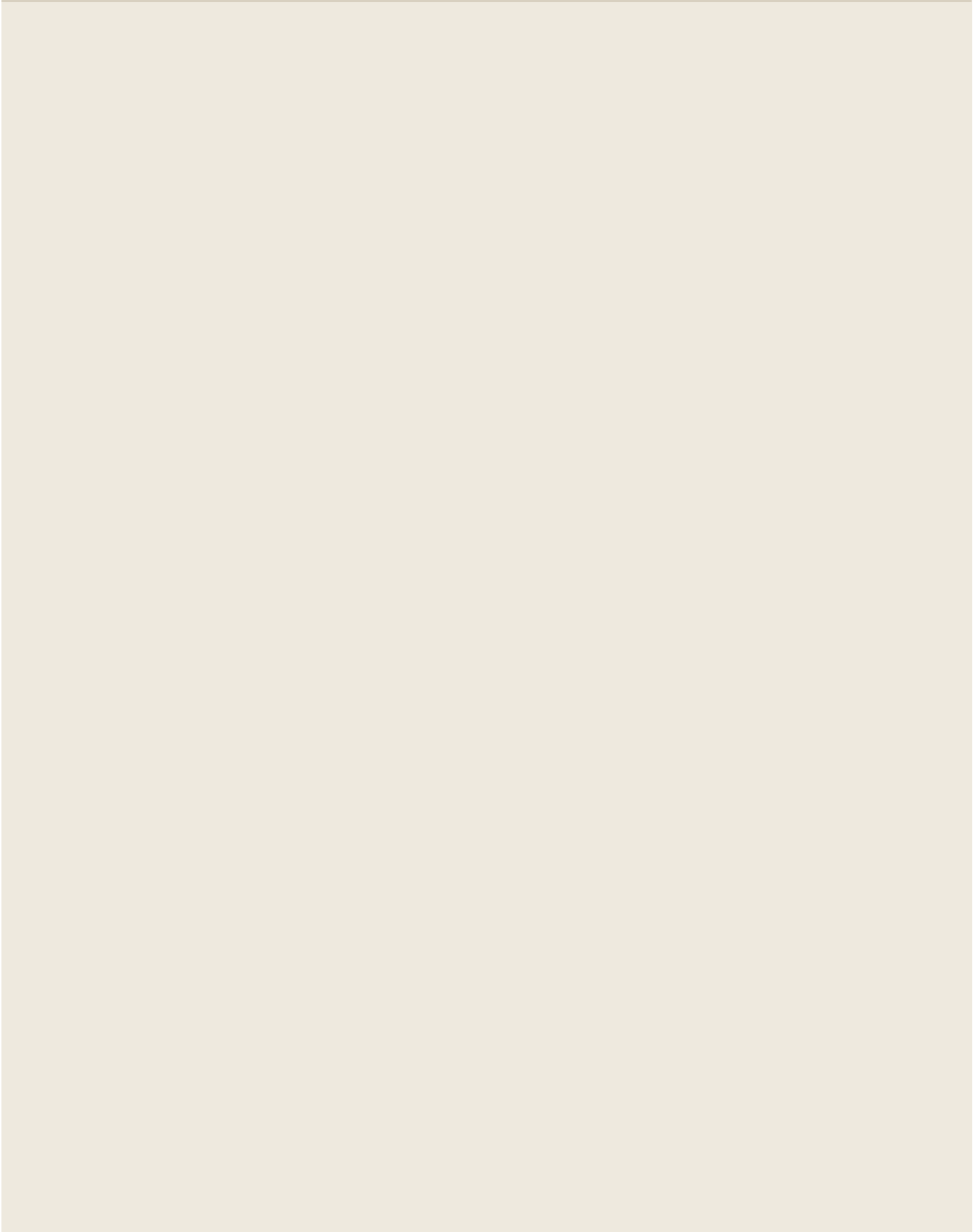
ARCHITECTURE

PACKAGE	ROLE
<code>core/</code>	Pure, deterministic reference model — capacity decode and conservation rules, no network or clock access.
<code>adapter/</code>	ProofBuilder HTTP client, viem-based precompile calls, proof assembly, append-only receipt ledger.
<code>cli/</code>	<code>notch verify</code> , <code>notch deployment-check</code> — run via <code>npx tsx cli/src/index.ts <command></code> .

contract **Solidity:** MockUSDC (ccUSD), AttestedCashflowRegistry, CashflowLendingVenue, NotchDemoAsset.
s/

web/ The Next.js dashboard — Verify, Position, Activity, Docs.

data/ Append-only receipt ledger (ledger.jsonl) and per-gate evidence.



Public addresses, both chains

CONTRACT	NETWORK	ADDRESS
AttestedCashflowRegistry	Creditcoin CC3 testnet	0x68952727aa2e684bec378120a651434b1de5e915
CashflowLendingVenue	Creditcoin CC3 testnet	0x5CEB2357eCC2fcA5eA77057ce8D74646E0C76493
MockUSDC (ccUSD)	Creditcoin CC3 testnet	0x92db42991210a8f5a4c1de2ad36f10ef239f3498
SablierLockup v4.0 (allowlisted source)	Ethereum Sepolia	0xe61cb9153356419bdaD0A8767c059f92d221a3C4
NotchDemoAsset (locked deposit token)	Ethereum Sepolia	0x7D20609BDB2fA772143905D6aDA079b1515db0b0

These are the single source of truth for addresses; `web/src/lib/constants.ts` resolves to the same values, confirmed by `notch deployment-check`.

INDEPENDENTLY VERIFYING CAPACITY YOURSELF

`available()` is always `originalCapacity - financedCapacity`, computed by the registry contract itself — no app-side arithmetic:

```
cast call 0x68952727aa2e684bec378120a651434b1de5e915 \
  "available(bytes32) (uint128)" <claimId> \
  --rpc-url https://rpc.cc3-testnet.creditcoin.network
```

The proof trail, one real position followed end to end

Sablier stream #189, Ethereum Sepolia, 100,000 deposit — currently active, unlocks 2026-10-12. Every hash below is a real, mined, independently checkable transaction.

1 · SOURCE — SEPOLIA, THE CAPACITY WAS DECODED FROM THIS TRANSACTION
0x1e7d304d9d8ed1bb4ef54961c2d5551b35c224403fa46de30bb9cb28144a81fa

2 · ACTIVATION — CREDITCOIN, INSTANTIATECLAIM() USING THE ATTESTCOIN PROOF, BLOCK 5,475,774
0x2f2ada0a4d1b0f0030227de7b464c94e723d524815b21c67dfed45362aefeffe

Registry state right after: `originalCapacity = 100,000`.

3 · FINANCED — CREDITCOIN, BLOCK 5,475,775 (THE VERY NEXT BLOCK) **SUCCESS**
0x442e69ef2ca5122d686834e1db867a587bc23ca35178bdd1d9d0c4a541fb8978

70,000 ccUSD transferred; `available()` read live at block 5,475,774 (before) = **100,000**; at block 5,475,775 (this block) = **30,000**. Both figures independently re-read at those exact historical blocks, not inferred.

4 · REFUSED — CREDITCOIN, BLOCK 5,475,776 (THE BLOCK RIGHT AFTER) **REVERTED**
0x62cb553555d429951b88b9fc38459b1f57cdc2225ee0a1254edece662a4a8b93

A second, independent lender requested 50,000 against the 30,000 that remained. Mined and reverted, not a preview. Contract error: `InsufficientFinancingCapacity()` — raw selector `0xa3388671`, independently re-computable as `keccak256("InsufficientFinancingCapacity()").slice(0,4).available()` stayed **30,000** immediately after — the revert changed nothing, as expected.

CLAIMID

0x3fed0d1620134bb777847debb9f2bb9f7c455668f67b03c780f0f6896c0a0a48

"CREATE A DEMO CASHFLOW" — THE IN-APP FEATURE THAT LETS A JUDGE MINT THEIR OWN STREAM, PROVEN WORKING

Real Sablier stream #190 created end to end by the exact same code path the UI uses, exercised once to confirm it (not a mock):

APPROVE (NOTCHDEMOASSET → SABLIER)
0x6319689eeae58c4362b75f70170da307a9012abca92ecf93cb099b2b577658

CREATE (CREATEWITHTIMESTAMPSLL), BLOCK 11,695,944 — STREAM ID 190 PARSED FROM THE REAL EVENT,
NEVER GUESSED

0xf5904f24e8980a424346478b71590901435cc78646fe5f45f4716a3541a55260

Conservation, measured — not just claimed

3,280

units of over-financing
prevented

87/87

automated tests passing

44/44

ledger receipts MATCH

Frozen, real, 5-position measurement campaign — full shared conservation vs. a per-lender-ceiling baseline, same real decoded capacities, same fixed draw plan, on these deployed contracts. Denominator: 5 positions × 4 draw steps each × 3 funded lenders. The frozen-set hash was committed to `GATES.md` before any finance scenario ran, and every receipt id and the full ablation table are recorded there.

POSITION	SEPOLIA STREAM	DEPOSIT
1	181	500
2	182	800
3	183	1,200
4	184	600
5	185	1,000

VERIFY IT YOURSELF, INDEPENDENT OF THIS APP

```
npx tsx cli/src/index.ts verify <receipt_id>
node --import tsx scripts/verify-all-receipts.ts # every receipt in the ledger, recomputed from the pure reference model
```

Recomputes each receipt independently from its own recorded proof inputs via the pure reference model (`core/src/instantiate.ts` / `core/src/apply.ts`) — not this app's own arithmetic.

What is proven, what is not, and where this sits

SCOPE & BOUNDARIES

This build proves capacity conservation and atomic, real-time origination. It does not provide repayment, collateral recovery, or outbound writes back to the source chain — Attestcoin's current release is read-only, Ethereum to Creditcoin, and repayment is v2 scope. All tokens involved (the deposit asset, ccUSD) are testnet-only with no cash value.

THIRD-PARTY IP

Every contract in `contracts/src/` is original to this repo. Nothing here forks or vendors another team's codebase. Official Gluwa/Creditcoin repositories (precompile ABIs, `EvmV1Decoder.sol`) were read as public interface references during development — never copied wholesale, never run at build or runtime. `@gluwa/usc-sdk` is deliberately not a dependency anywhere in this repo (it pulls in `ethers`, which this project avoids entirely); Sablier and Creditcoin's own precompiles are called directly, by address, against their published interfaces.

WHERE THIS SITS AGAINST THE FIELD

This season's Attestcoin submissions largely share one shape: prove an Ethereum fact, then make a single lending or scoring decision from it — Attestcoin's own Tutorial 4 (a cross-chain loan dApp), CrossCredit (proves Aave repayments and lends), CreditX (proves Ethereum history, mints a score, lends), and the prior season's HashCredit (proved an external income stream, originated one loan). Each treats the proven fact as information read once by one decision. Under the same positions, lenders and amounts, that shape cannot represent how much of a position is *already* spent by someone else — independent per-lender guards can't see each other, so they permit collective over-allocation that a shared, conserved capacity refuses by construction. That is the wedge Notch is built around: not a better proof, but a shared ledger that remembers what every other lender already drew.

ROADMAP (V2, NOT CLAIMED HERE)

- Repayment and collateral recovery flows.
- Outbound writes back to the source chain, once Attestcoin's writability subsystem supports it.
- Additional source protocols beyond Sablier Lockup, behind the same fail-closed shape checks.

Everything above, checkable directly

RESOURCE	LOCATION
Live app	https://notch-web-sooty.vercel.app/ — no wallet needed to browse, zero environment variables
Source repository	https://github.com/TheWeirdDee/notch (MIT license)
Full integration doc	<code>ATTESTCOIN_INTEGRATION.md</code> — verify/decode/instantiate path, guarantees table
Every gate, every tx	<code>GATES.md</code> — the complete build history with evidence
Implementation decisions	<code>DECISIONS.md</code> — append-only, corrections recorded not silent
In-app documentation	<code>/docs</code> route — the same integration depth, browsable

Notch enforces one shared financing capacity on Creditcoin. In its live CC3 testnet demonstration, a 100,000-capacity claim accepted a 70,000 ccUSD financing transaction, leaving 30,000 available. A subsequent lender attempted to finance another 50,000 against the same claim and the transaction reverted on-chain. Every number above can be recomputed directly from the Creditcoin RPC without trusting this document or this app's frontend.